

Constructors in Java

01 May 2026 06:51

In Java, a constructor is a special method used to initialize object, when a class is created.

What is a Constructor in Java?

A constructor in Java is a block of code like the method. It is called when an instance of a class is created. At the time of calling constructor, memory for the object is allocated in the memory. It is a special type of method which is used to initialize the object.

Every time when we create an object by using the new keyword. It calls a default constructor. If there is no constructor available in the class. In such case Java compiler provides a default constructor by default.

Note: It is called constructor because it constructs the values at the time of object creation. It is not necessary to write a constructor for a class because Java compiler creates a default constructor if a class does not have any.

Rules for Creating Java Constructor

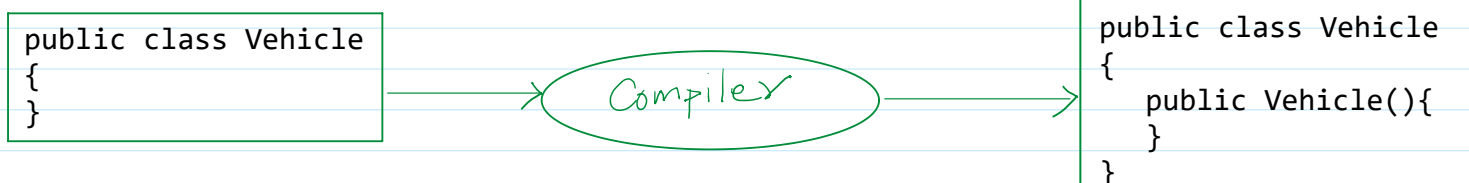
1. Constructor name must be the same as its class name.
2. The constructor must have no explicit return type.
3. A constructor cannot be abstract, static, final, or synchronized.

Note: We can use access modifiers while declaring a constructor. It controls the object creation. A constructor may be private, protected, public, or default.

Types of Java Constructors

There are two types of constructors in Java:

1. **Default Constructor(No-arg constructor):** The default constructor does not take any parameters and is used to initialize an object with default values. If no constructor is defined, Java automatically provides a default constructor.
2. **Parameterized Constructor:** The parameterized constructor accepts the parameters and is used to initialize an object with specific values at the time of object creation.
 - a. **Copy constructor:** A copy constructor is a special type of constructor that is commonly utilized to create a new object by copying the values of an existing object of the same class.



Purpose of a default constructor

The default constructor is used to provide the values to the object like zero, null etc. depending on the type.

package OOP;

```

/**
 * Write a description of class Vehicle here.
 *
 * @author (your name)
 * @version (a version number or a date)
 */
public class Vehicle
{
    //instance variables
    int engineNum;
    String model;
    double price;

    public void display(){
        System.out.print("\nEngine number: " + engineNum
            + "\n Model: " + model+
            "\nPrice; " + price);
    }
}

```

Output:

```

Engine number: 0
Model: null
Price: 0.0

```

Java Copy Constructor(type of parametrized constructor)

There are the following three ways to copy the values of one object into another:

- Using a Constructor
- By Assigning the Values of one Object to Another
- By using clone() Method of the Object class.

```
package OOP;
```

```

/**
 * Write a description of class Student here.
 *
 * @author (your name)
 * @version (a version number or a date)
 */
public class Student implements Cloneable
{
    private int rollNumber;
    private String name;
    private byte std;

    public Student(){ //default constructor defined by the user
        rollNumber = 0;
    }
}

```

```

        name = "";
        std = 0;
    }

    //parametrized constructor, used to initialize instance variable of object
    public Student(int rn, String nm, byte std){
        this.rollNumber = rn;
        this.name = nm;
        this.std = std;
    }

    public Student(Student st){    //copy constructor
        this.rollNumber = st.rollNumber;
        this.name = st.name;
        this.std = st.std;
    }

    public void setRollNumber(int rollNumber){
        this.rollNumber = rollNumber;
    }

    public void setName(String nm){
        this.name = nm;
    }

    public void setStd(byte std){
        this.std = std;
    }

    public int getRollNumber(){
        return this.rollNumber;
    }

    public String getName(){
        return this.name;
    }

    public byte getStd(){
        return this.std;
    }

    // Overriding the clone() method
    @Override
    public Object clone() throws CloneNotSupportedException {
        // Returning a clone of the current object
        return super.clone();
    }

    //factory method
    public static Student newStudent(){
        return new Student();
    }

```

```

        @Override
        public String toString(){
            return "\nRoll number: " + rollNumber +
                "\nName " + name + "\nStd: " + std;
        }
    }

package OOP;
import java.util.*;

/**
 * Write a description of class CreateStudents here.
 *
 * @author (your name)
 * @version (a version number or a date)
 */
public class CreateStudents
{
    public static void main(String[] args) throws CloneNotSupportedException {
        Scanner sc = new Scanner(System.in);
        System.out.print("\f");
        Student s1 = new Student(); //by calling default constructor
        System.out.print("Enter roll number: ");
        s1.setRollNumber(sc.nextInt());
        sc.nextLine(); //to remove the keyboard buffer
        System.out.print("Enter name of student: ");
        s1.setName(sc.nextLine());
        System.out.print("Enter standard of student: ");
        s1.setStd(sc.nextByte());

        //Another object of student
        // using parameterized constructor to initialize it
        System.out.print("\nEnter roll number, name, and standard of student:
\n");
        int r = sc.nextInt();
        sc.nextLine();
        String nm = sc.nextLine();
        byte std = sc.nextByte();
        Student s2 = new Student(r, nm, std);

        System.out.print("\nStudent 1 info:");
        System.out.print("\nRoll number: " + s1.getRollNumber());
        System.out.print("\nName: " + s1.getName());
        System.out.print("\nStandard: " + s1.getStd());

        System.out.print("\nStudent 2 info:");
        System.out.print("\nRoll number: " + s2.getRollNumber());
        System.out.print("\nName: " + s2.getName());
        System.out.print("\nStandard: " + s2.getStd());

        try

```

```

{
    try
    {
        Student s3 = (Student)Class.forName("Student").newInstance();
    }
    catch (InstantiationException ie)
    {
        ie.printStackTrace();
    }
}
catch (IllegalAccessException iae)
{
    iae.printStackTrace();
}
catch(ClassNotFoundException ob){

}

Student st5 = (Student)s1.clone();

Student st6 = Student.newStudent();

Student st7 = new Student(st6); //Using Copy Constructor

//Changed the state of st7, which is the copy of st6
st7.setStd((byte)10);

System.out.print("\n" + st6 + "\n" + st7);
}
}

```

Output:

```
BlueJ: Terminal Window - Kapish11
Options
Enter roll number: 1
Enter name of student: Kapish
Enter standard of student: 11

Enter roll number, name, and standard of student:
2
Grish
11

Student 1 info:
Roll number: 1
Name: Kapish
Standard: 11
Student 2 info:
Roll number: 2
Name: Grish
Standard: 11

Roll number: 0
Name
Std: 0

Roll number: 0
Name
Std: 10
```

Constructor Chaining

Constructor chaining in Java is a practice where one constructor calls another constructor of the same class or a superclass during object creation. It is typically done using either **this()** to call another constructor in the same class, or **super()** to call a constructor in the Super class. By centralizing common constructor logic, constructor chaining helps reduce code redundancy and increases the code readability.

In Java, constructor chaining is a sequence of invoking constructor where one constructor calls another constructor within the same class or from its superclass by using an instance.

Types of Constructor Chaining

We can achieve constructor chaining in the following two ways:

- **Within the same class:** If the constructor belongs to the same class, we use **this()** to call another constructor in the same class.
- **From the base class:** If the constructor belongs to different classes (parent and child classes) We use the **super()** to call the constructor from the base or parent class.

For example:
package OOP;

```
/**
 * Write a description of class Student here.
 *
 * @author (your name)
 * @version (a version number or a date)
```

```

*/
public class Student implements Cloneable
{
    private int rollNumber;
    private String name;
    private byte std;
    //Constructor Chaining
    public Student(){ //default constructor defined by the user
        this(0,"",(byte)0); //calling parametrized constructor
    }

    //parametrized constructor, used to initialize instance variable of object
    public Student(int rn, String nm, byte std){
        this.rollNumber = rn;
        this.name = nm;
        this.std = std;
    }

    public Student(Student st){ //copy constructor
        this(st.rollNumber, st.name, st.std); //calling parametrized constructor
    }

    public void setRollNumber(int rollNumber){
        this.rollNumber = rollNumber;
    }

    public void setName(String nm){
        this.name = nm;
    }

    public void setStd(byte std){
        this.std = std;
    }

    public int getRollNumber(){
        return this.rollNumber;
    }

    public String getName(){
        return this.name;
    }

    public byte getStd(){
        return this.std;
    }

    // Overriding the clone() method
    @Override
    public Object clone() throws CloneNotSupportedException {
        // Returning a clone of the current object
        return super.clone();
    }
}

```

```

        //factory method
        public static Student newStudent(){
            return new Student();
        }

        @Override
        public String toString(){
            return "\nRoll number: " + rollNumber +
                "\nName " + name + "\nStd: " + std;
        }
    }

package OOP;
import java.util.*;

/**
 * Write a description of class CreateStudents here.
 *
 * @author (your name)
 * @version (a version number or a date)
 */
public class CreateStudents
{
    public static void main(String[] args) throws CloneNotSupportedException {
        Scanner sc = new Scanner(System.in);
        System.out.print("\f");
        Student s1 = new Student(); //by calling default constructor
        System.out.print("Enter roll number: ");
        s1.setRollNumber(sc.nextInt());
        sc.nextLine(); //to remove the keyboard buffer
        System.out.print("Enter name of student: ");
        s1.setName(sc.nextLine());
        System.out.print("Enter standard of student: ");
        s1.setStd(sc.nextByte());

        //Another object of student
        // using parameterized constructor to initialize it
        System.out.print("\nEnter roll number, name, and standard of student:
\n");
        int r = sc.nextInt();
        sc.nextLine();
        String nm = sc.nextLine();
        byte std = sc.nextByte();
        Student s2 = new Student(r, nm, std);

        System.out.print("\nStudent 1 info:");
        System.out.print("\nRoll number: " + s1.getRollNumber());
        System.out.print("\nName: " + s1.getName());
        System.out.print("\nStandard: " + s1.getStd());

        System.out.print("\nStudent 2 info:");
    }
}

```



```

System.out.print("\nRoll number: " + s2.getRollNumber());
System.out.print("\nName: " + s2.getName());
System.out.print("\nStandard: " + s2.getStd());

try
{
    try
    {
        Student s3 = (Student)Class.forName("Student").newInstance();
    }
    catch (InstantiationException ie)
    {
        ie.printStackTrace();
    }
}
catch (IllegalAccessException iae)
{
    iae.printStackTrace();
}
catch(ClassNotFoundException ob){

}

Student st5 = (Student)s1.clone();

Student st6 = Student.newStudent();

Student st7 = new Student(st6); //Using Copy Constructor

//Changed the state of st7, which is the copy of st6
st7.setStd((byte)10);

System.out.print("\n" + st6 + "\n" + st7);
}
}

```

Output:

```
BlueJ: Terminal Window - Kapish11
Options
Enter roll number: 1
Enter name of student: Kapish
Enter standard of student: 11

Enter roll number, name, and standard of student:
2
Harsh
11

Student 1 info:
Roll number: 1
Name: Kapish
Standard: 11
Student 2 info:
Roll number: 2
Name: Harsh
Standard: 11

Roll number: 0
Name
Std: 0

Roll number: 0
Name
Std: 10
```

The code has reduced, generates the same output.

Difference between Constructor and Method in Java

Java Constructor	Java Method
A constructor is used to initialize the state of an object.	A method is used to expose the behavior of an object.
A constructor must not have a return type.	A method must have a return type.
The constructor is invoked implicitly.	The method is invoked explicitly.
The Java compiler provides a default constructor if we do not have any constructor in a class.	The method is not provided by the compiler in any case.
The constructors name must be the same as the class name.	The method name may or may not be same as the class name.

Homework

1. Custom Constructors & Object Tracking

Create a class Book that stores title, author, price, and a unique book ID auto-generated for each new object.

Requirements:

- Implement multiple constructors: default, one with title+author, and one with all fields.
- Maintain a static counter that increments for each book created.
- Add a method applyDiscount(double percent).
- Write a demo program that creates several books using different constructors and prints all information.

2. Constructor Overloading with Validation

Build a Student class with fields: name, age, and grade.

Requirements:

- Provide three constructors: default, name-only, and full constructor.
- Validate that age is between 5-120, otherwise default to 18.
- Validate grade range 0.0-4.0, otherwise default to 0.0.
- Include a printDetails() method.

Create a tester that demonstrates all cases (valid and invalid).

3. Chaining Constructors (this())

Create a class Rectangle with width and height.

Requirements:

- Use constructor chaining so the default constructor calls the one-argument constructor, which then calls the two-argument constructor.
- Add methods for area, perimeter, and resizing the rectangle.
- Show how each constructor is triggered with a test program.

4. Bank Account Simulation with Multiple Constructors

Create a BankAccount class with: name, account number, and balance.

Requirements:

- Constructor that auto-generates the account number.
- Another constructor allows custom account number (for migrating customers).
- Methods: deposit, withdraw (with insufficient balance warning), transfer.
- A driver class that simulates multiple operations between accounts.

5. Inventory System Using Array of Objects

Write a Product class (id, name, quantity, price) with overloaded constructors.

Write a second class Inventory that uses an array of Product objects.

Requirements:

- Add a method to add new products, update quantities, compute total inventory value, and search by name.
- Demonstrate your system with at least five products.

6. Library Management with Constructors and Encapsulation

Create classes Author and Book.

Requirements:

- Author has name, email, and gender.
- Book has title, price, and an Author object.
- Implement constructors for both classes.
- Add methods to display complete book info including author details.

Create a main program that builds several books using different authors.

7. Object Copying (Copy Constructor Scenario)

Create a class Point (x, y).

Requirements:

- Implement a constructor that initializes x and y.
- Implement a copy constructor that copies coordinates from another Point object.
- Add a method to move the point by dx, dy.
- In main, create a point, create its copy, modify the original, and show how the copy behaves.

8. Employee Payroll with Constructor Logic

Create an Employee class with name, hourlyRate, and hoursWorked.

Requirements:

- Overload constructors: default, name-only, name+rate.
- Pay should be computed using a method that includes overtime (hours > 40 paid at 1.5x).
- Add a method to print a formatted pay slip.
- Demonstrate using different employees.